



School of Future Tech

Case Study Report

on

**Case Study 143: Smart Waste Collection & Recycling Management
System**

by

Ankitraj Jha

150096725050

Case Study: Smart Waste Collection & Recycling Management System

Index

1. Introduction to the Case Study.
 2. Problem Statement / Case Background (Abstract).
 3. Problem Statement / Case Study Design.
 4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
 5. Problem Statement / Case Study Implementation Details and Snapshots.
 6. Problem Statement / Case Study Results and Conclusion.
 7. References
-

1. Introduction to the Case Study

As modern municipalities undergo rapid urbanization, the volume of solid waste generated daily escalates exponentially. Municipal corporations are tasked with coordinating collection routines under constraints of restricted budgets, limited fleets, fuel cost fluctuations, and environmental goals.

Traditional municipal solid waste management systems suffer from high operational overhead due to fixed schedules and static routing. Collection vehicles visit zones without knowing bin fill status, leading to empty trips or overflows. Furthermore, citizens complain of long response times for service requests, and managing databases of thousands of bins becomes slow if search indexes degrade.

This case study designs and implements a **Smart Waste Collection & Recycling Management System** in C++ using custom-designed Data Structures and Algorithms. The platform integrates IoT sensor simulation, shortest-path road navigation, dynamic priority queues, and self-balancing tree databases to provide a highly scalable, real-time command dashboard.

Efficient algorithmic waste management is applicable in:

Smart Cities: Municipal route optimization and sensor-driven garbage collection.

Logistics & Fleet Management: Real-time asset checks, license lookups, and scheduling.

Environmental Sustainability: Dynamic estimation of recyclable volumes and resource deployment.

2. Problem Statement (Abstract)

Managing urban waste requires scheduling vehicles to clean bins across different districts (commercial, industrial, residential) and handling citizen service tickets. The primary technical challenges include:

1. **Inefficient Scheduling:** Sorting collection orders randomly or by static location causes bins with high waste levels to overflow, generating odors and hazards. Bins must be sorted dynamically by priority scores based on fill percentage and capacity.

2. **Logarithmic Retrieval Bounds:** Inserting sequential bin IDs into standard binary search trees causes skewing, degrading lookup times from $O(\log N)$ to $O(N)$ linear scans. Database structures must maintain balance factors.
 3. **Logistics Route Optimization:** Manually routing collection vehicles leads to excessive fuel consumption. A graph representation of city intersections must be queried to calculate the shortest path.
 4. **Log Retrieval & Undo Actions:** Operational mistakes (such as duplicate entries or incorrect routing) require a recovery mechanism that lists and rolls back changes in LIFO order.
 5. **Fair Ticket Handling:** Citizen complaints must be processed in a strict FIFO manner to prevent long wait times.
-

3. Case Study Design

The smart waste platform is designed as an interactive command-line dashboard that operates entirely on custom-built pointer structures, replacing STL dependencies to demonstrate low-level DSA logic.

Input

Waste Bin Array: Dynamic entries representing bins with:

- ID (integer key)
- Location (string name)
- Fill Level (0 to 100%)
- Capacity (in liters)
- Waste Type (Organic, General, Recyclable, Hazardous)

City Intersections Map: A weighted, undirected graph network consisting of 10 vertices (Central Depot, Recycling Center, and key zones) and 12 weighted edges (distances in km).

Fleet Vehicles: License plate strings, driver names, truck type, and status flags.

Service Tickets: Citizen name, location, problem description, and urgency ratings.

Constraints

Memory Bounds: Explicit deletion of dynamically allocated tree nodes, queue items, and stack items to prevent leaks.

Sequential Sorting Constraint: Prioritization must run sorting operations in-place without standard templates (`std::sort`).

Balanced Trees Constraint: Tree height must remain bounded to $O(\log N)$ under sequential insertions.

Real-World Mapping

Ultrasonic Sensors: Send fill levels dynamically to update bin records.

License Plates: Act as unique keys for polynomial hashing to track active trucks.

Graph Vertices: Depot, zones, and transfer stations.

4. Methods & Algorithms Technology Applied

Feature	Data Structure / Algorithm	Justification
Collection Scheduling	Sorting Algorithms (Merge/Quick)	Orders the collection zones by waste generation level to prioritize high-need areas efficiently.
Record Retrieval	Searching Algorithms (Binary Search)	Allows quick, logarithmic-time lookup of historical recycling and collection records from sorted databases.
Recovery Procedures	Stack	Uses LIFO (Last-In-First-Out) ordering. The most recent operational change is at the top, perfectly mimicking the behavior of an 'Undo' button.
Service Requests	Queue	Enforces FIFO (First-In-First-Out) processing, guaranteeing that citizen or municipal requests are processed strictly in the order they were submitted.
Asset Identification	Hash Table	Provides near-instant ($O(1)$) lookup times to verify if a given physical asset (RFID or GPS ID) already exists or to find its status.
Waste Database	BST / AVL Trees	Automatically balances the collection database so that insertion, deletion, and searching remain fast and predictable at $O(\log N)$.
Route Optimization	Graph & Dijkstra's Algorithm	Models city intersections as vertices and roads as edges with distance weights. Dijkstra's guarantees the shortest and most cost-effective path for garbage trucks.

Time & Space Complexity

Component	Operation	Time Complexity	Space Complexity
Sorting Algorithms	Sort Schedules	$O(N \log N)$	$O(N)$ or $O(\log N)$
Searching Algorithms	Record Lookup	$O(\log N)$	$O(1)$
Stack	Push / Pop	$O(1)$	$O(A)$ where A is number of stored actions

Component	Operation	Time Complexity	Space Complexity
Queue	Enqueue / Dequeue	$O(1)$	$O(S)$ where S is requests in queue
Hash Table	Insert / Lookup	$O(1)$ average, $O(C)$ worst	$O(C)$ where C is unique assets
BST/AVL Tree	Insert / Search	$O(\log D)$	$O(D)$ where D is collection points
Graph	Add Edge	$O(1)$ average	$O(V + E)$ where V=intersections, E=roads
Dijkstra / BFS	Route finding	$O((V + E) \log V)$	$O(V)$ for priority queue and visited sets

5. Problem Statement / Case Study Implementation Details and Snapshots

The core logic of the system is written in `smart_waste_system.cpp`. The platform avoids pre-built standard templates, creating custom objects for the data layers.

Core Structure Implementations

1. Custom Singly-Linked Stack (LIFO Recovery Log)

The recovery mechanism logs all operational modifications using dynamic singly linked nodes:

2. Custom FIFO Queue (Service Request Manager)

Citizen tickets are processed in order using a linked queue structure tracking head and tail nodes:

3. Custom AVL Self-Balancing Tree (Balanced Database Storage)

Primary database storage inherits standard binary search operations but wraps them in rotations to maintain logarithmic height:

None

```
struct TreeNode {
    WasteBin bin;
    TreeNode *left, *right;
    int height;
};
```

```

class AVL {
public:
    TreeNode* root = nullptr;
    int count = 0;
    int getH(TreeNode* n) { return n ? n->height : 0; }
    int getBF(TreeNode* n) { return n ? getH(n->left) -
getH(n->right) : 0; }
    void updH(TreeNode* n) { if (n) n->height = 1 +
maxVal(getH(n->left), getH(n->right)); }

    TreeNode* rotR(TreeNode* y) {
        TreeNode* x = y->left, *T2 = x->right;
        x->right = y;
        y->left = T2;
        updH(y);
        updH(x);
        return x;
    }
    TreeNode* rotL(TreeNode* x) {
        TreeNode* y = x->right, *T2 = y->left;
        y->left = x;
        x->right = T2;
        updH(x);
        updH(y);
        return y;
    }
    TreeNode* bal(TreeNode* n) {
        updH(n);
        int bf = getBF(n);
        if (bf > 1) {
            if (getBF(n->left) < 0) n->left = rotL(n->left);
            return rotR(n);
        }
        if (bf < -1) {
            if (getBF(n->right) > 0) n->right = rotR(n->right);
            return rotL(n);
        }
        return n;
    }
}

```

```

    }
    TreeNode* ins(TreeNode* n, WasteBin b) {
        if (!n) { count++; return new TreeNode{b, nullptr,
        nullptr, 1}; }
        if (b.id < n->bin.id) n->left = ins(n->left, b);
        else if (b.id > n->bin.id) n->right = ins(n->right, b);
        else return n;
        return bal(n);
    }
    void insert(WasteBin b) { root = ins(root, b); }
};

```

4. Custom Hash Table with Collision Chaining (License lookup)

Fleet assets are indexed in a size-53 bucket structure using a polynomial rolling hash:

5. Shortest Route Optimization (Dijkstra's Pathfinder)

Computes shortest paths on the city graph using relaxation over arrays:

None

```

void dijkstra(int src, int dst) {
    int dist[maxVertices], prev[maxVertices];
    bool visited[maxVertices];
    for (int i = 0; i < V; i++) {
        dist[i] = 999999;
        prev[i] = -1;
        visited[i] = false;
    }
    dist[src] = 0;
    for (int count = 0; count < V - 1; count++) {
        int minDist = 999999, u = -1;
        for (int v = 0; v < V; v++) {
            if (!visited[v] && dist[v] < minDist) {
                minDist = dist[v];
                u = v;
            }
        }
        if (u == -1) break;
        visited[u] = true;
        for (size_t i = 0; i < adjList[u].size(); i++) {
            int neighbor = adjList[u][i].dest;

```

```

        int weight = adjList[u][i].weight;
        if (!visited[neighbor] && dist[u] + weight <
dist[neighbor]) {
            dist[neighbor] = dist[u] + weight;
            prev[neighbor] = u;
        }
    }
}

```

6. Dynamic Priority Sorting (Heap Sort)

Orders bin indices in-place based on their current priority weight:

Orders bin indices in-place based on their current priority weight:

Executable Snapshots

1. Main System Dashboard

The system displays real-time statistics (total bins, urgency counts, pending citizen requests, recommended actions) and the selection menu:

```
+-----+  
| Smart Waste Collection & Recycling Management System |  
+-----+  
|  
| System Status  
| Total Bins : 11  
| Urgent Bins : 4 need collection  
| Pending Requests : 5 citizen issues  
| Registered Vehicles : 7  
| Recommended Action : Review Collection Planning  
|  
+-----+  
| [1] Waste Bin Management  
| [2] Collection Planning  
| [3] Citizen Service Requests  
| [4] Vehicle Management  
| [5] Route Planning  
| [6] Environmental Report  
| [7] Technical Analysis  
| [8] Recovery Log  
| [0] Exit  
|  
+-----+  
Enter Choice:
```

2. Waste Bin Records (Primary Storage AVL Tree)

Selecting option [1] -> [1] lists all registered bins in the database:

None

=== Waste Bin Records ===

ID	Location	Fill%	Capacity	Type
Status				
2	101	2%	3L	6
Normal				
101	Main Street	92%	500L	Organic
URGENT				
103	Industrial Area	88%	1000L	General
URGENT				
104	Residential North	30%	400L	Organic
Normal				
105	Residential South	76%	400L	
Recyclable	Moderate			
106	Market District	95%	600L	General
URGENT				
107	Commercial Hub	55%	750L	
Hazardous	Moderate			
108	University Area	40%	350L	
Recyclable	Normal			
109	Town Square	83%	450L	Organic
URGENT				
110	Riverside Zone	62%	550L	General
Moderate				
1003	mumbai	50%	7L	wet
Moderate				

Total Bins: 11 | Primary Storage Depth: 4

3. Collection Scheduling (Heap Sorted Priority List)

Selecting option [2] runs the custom Heap Sort algorithm to order bins descending by their Priority Score:

None

=== Collection Priority List ===

+-----+-----+-----+-----+-----				
-----+				
Priority	Bin ID	Location	Fill	
Action Required				
+-----+-----+-----+-----+-----				
-----+				
1	103	Industrial Area	88%	
Collect Now				
2	106	Market District	95%	
Collect Now				
3	101	Main Street	92%	
Collect Now				
4	107	Commercial Hub	55%	
Schedule Soon				
5	109	Town Square	83%	
Collect Now				
6	110	Riverside Zone	62%	
Schedule Soon				
7	105	Residential South	76%	
Schedule Soon				
8	108	University Area	40%	
Routine Check				
9	104	Residential North	30%	
Routine Check				
10	1003	mumbai	50%	
Schedule Soon				
11	2	101	2%	
Routine Check				
+-----+-----+-----+-----+-----				
-----+				

4. Shortest Route Finder (Dijkstra's Path)

Selecting option [5] -> [4] finds the shortest distance between Central Depot [0] and Recycling Center [9]:

None

Shortest Route from Central Depot to Recycling Center:
[Central Depot] --(3km)--> [Park Avenue] --(3km)--> [Residential South] --(2km)--> [University Area] --(3km)--> [Recycling Center]

Total Distance: 11 km

Distances from Central Depot to all areas:

Destination Area	Distance
Central Depot	0 km
Main Street	2 km
Park Avenue	3 km
Industrial Area	6 km
Residential North	4 km
Residential South	6 km
Market District	8 km
Commercial Hub	7 km
University Area	8 km
Recycling Center	11 km

5. Database Balancing Structure (AVL Tree Height Balancing)

Selecting Option [7] (Technical Analysis) -> Option [1] prints node details and connections:

None

=== Primary Storage Status (AVL Tree) ===
... Preorder Traversal: 108 104 101 2 103 106 105 107 110 109 1003

=== AVL Node Connections Table ===

Node ID	Left Child	Right Child
108	Bin 104	Bin 110
104	Bin 101	Bin 106
101	Bin 2	Bin 103
2	None	None

103	None	None	
106	Bin 105	Bin 107	
105	None	None	
107	None	None	
110	Bin 109	Bin 1003	
109	None	None	
1003	None	None	
+-----+	+-----+	+-----+	+

6. Database Storage Comparison (AVL vs BST Height)

Selecting option [7] -> [3] displays the difference in storage depth after sequential inserts:

None

=== Storage Comparison ===		
+-----+	+-----+	+
Storage Structure	Tree Height	
+-----+	+-----+	+
AVL Tree (Primary)	4	
BST Tree (Backup)	11	
+-----+	+-----+	+

7. Operational Recovery Log (Action Stack)

Selecting option [8] -> [1] returns chronological undo listings maintained on a custom Stack:

None

=== Recovery Log History ===	
+-----+	+-----+
-+	
No.	Action Description
+-----+	+-----+
-+	
1	Viewed technical analysis summary
2	Viewed Environmental Report
3	Planned route from Central Depot to Recycling Center

```
| 4      | Checked network connectivity from Central Depot
|
| 5      | Checked connections from Central Depot
|
| 6      | Registered vehicle GV-TRK-009
|
| 7      | Searched for vehicle plate: GV-TRK-001
|
| 8      | Processed request #2
|
| 9      | Submitted service request
|
| 10     | Viewed collection priority list
|
| 11     | Deleted Bin 111
|
| 12     | Searched for location: Market
|
| 13     | Searched Bin 103
|
| 14     | Inserted Bin 111
|
| 15     | Viewed waste bin records
|
| 16     | System initialized
|
+-----+-----+
-+
```

6. Problem Statement / Case Study Results and Conclusion

Analysis of Results

1. **Database Search Speeds:** The AVL tree successfully maintained a balanced height of 4 under successive insertions, whereas the standard BST skewed to a height of 11. This prevents search time degradation, guaranteeing logarithmic efficiency ($O(\log N)$) instead of linear scan delays.
2. **Transportation Savings:** Dijkstra's Algorithm computed an optimal path of 11 km from the Central Depot to the Recycling Center. Compared to standard routes, this reduces simulated fuel consumption and vehicle wear.

3. **Queue fairness:** Utilizing a custom singly-linked FIFO Queue ensures citizen requests are resolved in order, preventing ticket starvation.
4. **Undo Operations:** The Stack recovery log correctly tracks state changes, allowing undo rollbacks in constant $O(1)$ time.

Conclusion

The Smart Waste Collection & Recycling Management System demonstrates how custom pointer structures and algorithms can be integrated to resolve logistical, data storage, and administrative bottlenecks in modern smart cities.

For future extensions, the system can scale by:

IoT Sensor Integration: Feeding live ultrasonic bin readings directly to update the AVL tree nodes.

Live Traffic Weighing: Interfacing map APIs to adjust graph weights dynamically, reflecting congestion.

7. References

1. 1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. — *Introduction to Algorithms*, Third Edition, MIT Press.
2. 2. Knuth, D. E. — *The Art of Computer Programming, Volume 3: Sorting and Searching*.
3. 3. Dijkstra, E. W. — *A Note on Two Problems in Connexion with Graphs* (Numerische Mathematik, 1959).
4. 4. GeeksforGeeks — *AVL Trees, Graph Traversals, and Hashing Collision Resolutions*.
5. 5. Research paper: *IoT-based Smart Garbage Monitoring and Routing Optimization Systems for Smart Cities*.